

Aarya Arban

S11 – 07

Aim: Write a program for Merge sort using Object oriented language (Java, C++) and Haskell.

Java Code:

```
import java.util.*;

class MergeSort {

    public static void merge(int[] array, int[] new_array_1, int[] new_array_2, int left, int right)
    {
        int i = 0, j = 0, k = 0;

        while (i < left && j < right) {
            if (new_array_1[i] <= new_array_2[j]) {
                array[k++] = new_array_1[i++];
            } else {
                array[k++] = new_array_2[j++];
            }
        }

        while (i < left) {
            array[k++] = new_array_1[i++];
        }

        while (j < right) {
            array[k++] = new_array_2[j++];
        }
    }

    public static void mergeSort(int[] array, int length) {
        if (length < 2) {
            return;
        }
    }
}
```

```

int middle = length / 2;

int[] new_array_1 = new int[middle];

int[] new_array_2 = new int[length - middle];

for (int i = 0; i < middle; i++) {
    new_array_1[i] = array[i];
}

for (int i = middle; i < length; i++) {
    new_array_2[i - middle] = array[i];
}

mergeSort(new_array_1, middle);

mergeSort(new_array_2, length - middle);

merge(array, new_array_1, new_array_2, middle, length - middle);
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.println("Enter the size of the array: ");

    int size = sc.nextInt();

    int[] testScores = new int[size]; // Corrected the array declaration

    System.out.println("Enter the unsorted array: ");

    for (int i = 0; i < size; i++) {
        testScores[i] = sc.nextInt();
    }

    sc.close();

    System.out.println("Original Array " + Arrays.toString(testScores) + "\n");

    mergeSort(testScores, size);

    System.out.println("After Merge Sort " + Arrays.toString(testScores) + "\n");
}
}

```

Output:

Enter the size of the array:

4

Enter the unsorted array:

44

69

22

80

Original Array [44, 69, 22, 80]

After Merge Sort [22, 44, 69, 80]

C++ Code:

```
#include <iostream>

using namespace std;

void merge(int *arr, int left, int mid, int right) {
    int temp_left[mid - left + 1];
    int temp_right[right - mid];
    for (int i = 0; i < mid - left + 1; i++) {
        temp_left[i] = arr[left + i];
    }
    for (int i = 0; i < right - mid; i++) {
        temp_right[i] = arr[mid + 1 + i];
    }
    int i = 0;
    int j = 0;
    int k = left;
    while (i < mid - left + 1 && j < right - mid) {
        if (temp_left[i] <= temp_right[j]) {
            arr[k] = temp_left[i];
            i++;
        }
```

```

    } else {
        arr[k] = temp_right[j];
        j++;
    }
    k++;
}
while (i < mid - left + 1) {
    arr[k] = temp_left[i];
    i++;
    k++;
}
while (j < right - mid) {
    arr[k] = temp_right[j];
    j++;
    k++;
}
}

void merge_sort(int *arr, int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        merge_sort(arr, left, mid);
        merge_sort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    // Create an unsorted array.
    int arr[] = {50, 23, 12, 91, 4};
    merge_sort(arr, 0, sizeof(arr) / sizeof(arr[0]) - 1);

```

```

for (int i = 0; i < sizeof(arr) / sizeof(arr[0]); i++) {
    cout << arr[i] << " ";
}
cout << endl;
return 0;
}

```

Output:

4 12 23 50 91

Haskell Code:

```

mergelst :: [Int]->[Int]->[Int]->[Int]
mergelst [] [] z = reverse z
mergelst (x:xs) [] z = mergelst xs [] (x:z)
mergelst [] (y:ys) z = mergelst [] ys (y:z)
mergelst (x:xs) (y:ys) z = if x <= y then mergelst xs (y:ys) (x:z) else mergelst (x:xs) ys (y:z)
main = print( mergelst [ 10,30,69] [11,56,93,40] [] )

```

Output:

```

[1 of 1] Compiling Main          ( main.hs, main.o )
Linking main ...
[10,11,30,40,56,69,93]

```